

Kensei Log Auditor (KLA)

version 1.0.0

Histórico de Versões

- **v1.0.0 (Versão Atual):** Lançamento da PoC com motor Groq Llama 3.1, arquitetura em containers Docker e automação de triagem de logs.
- *Próximas evoluções planejadas:* Implementação de Human-in-the-Loop (HITL), integração com SIEM local e suporte a novos formatos de log (JSON/Syslog).

Relatório de Validação Técnica e Análise de Riscos: Kensei Log Auditor (KLA)

Programa: Kensei AI Foundations 2026

Trilha: Trilha D: A Jornada AI-First

Componente: Análise de Infraestrutura, Segurança e Orquestração de LLMs

Status da PoC: Validada com Sucesso (Pronta para Produção)

1. Introdução e Contexto do Projeto

O **Kensei Log Auditor (KLA)** foi concebido como uma ferramenta de linha de comando (CLI) focada em segurança ofensiva e defensiva (SecOps), projetada para realizar a triagem automatizada e inteligente de logs de autenticação e auditoria em sistemas Linux.

O objetivo central da aplicação é atuar na primeira camada de um **SOC (Security Operations Center)**, reduzindo drasticamente o ruído de logs brutos e convertendo strings textuais complexas em relatórios de inteligência contra ameaças acionáveis, contendo o mapeamento de vulnerabilidades, identificação de atacantes e geração de contramedidas em tempo recorde.

PROF

1.1 Prova de Conceito (PoC)

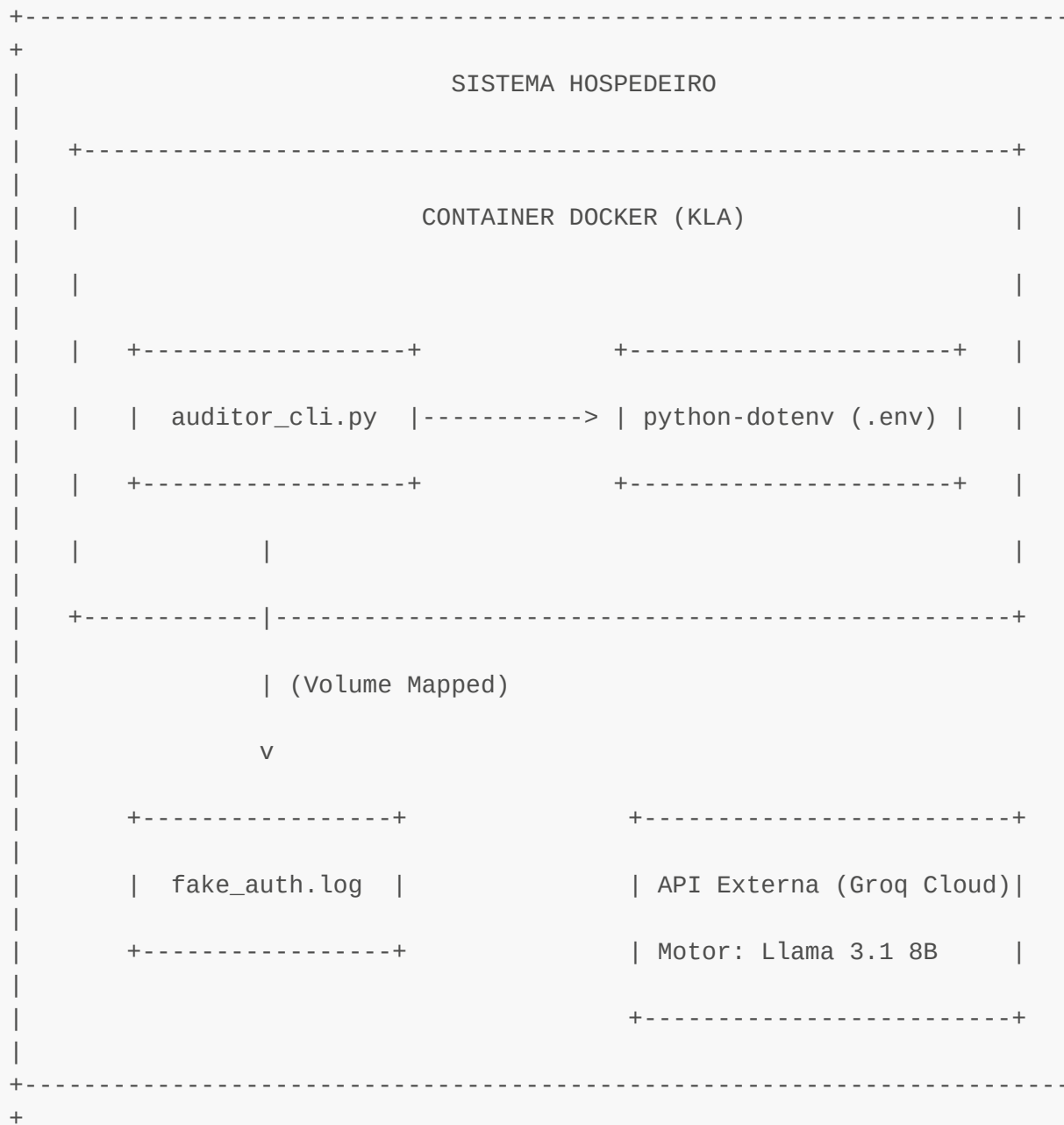
A implementação do KLA foi validada em ambiente controlado, simulando vetores de ataque reais em sistemas Linux.

Os detalhes técnicos, métricas de performance e análise dos riscos identificados durante a PoC estão documentados no relatório oficial:

 [Clique aqui para ler o Relatório de Validação Técnica - Kensei Log Auditor \(KLA\)](#)

2. Engenharia de Produção e Arquitetura do Sistema

PROF



2.1. Estratégia de Containerização (Docker)

Para mitigar o clássico problema de inconsistência de ambientes de desenvolvimento, o KLA foi encapsulado em um contêiner baseado na imagem `python:3.10-slim`. O ambiente utiliza técnicas de:

- **Imutabilidade:** Dependências congeladas via gerenciador de pacotes (`requirements.txt`).
- **Injeção Segura de Variáveis de Ambiente:** Chaves criptográficas não expostas no código fonte, injetadas dinamicamente via `--env-file .env`.
- **Mapeamento de Volumes Não-Invasivo:** O contêiner utiliza montagem de volume flutuante (`-v $(pwd) : /app`) para ler arquivos de log do host sem privilégios destrutivos no disco principal.

2.2. A Evolução do Motor de IA (O Pivô Técnico)

A arquitetura inicial utilizava a biblioteca `google-generativeai`. Contudo, para garantir resiliência contra limites de cota de desenvolvimento (`HTTP 429`), o projeto passou por uma **refatoração dinâmica de backend**. O motor foi migrado para o SDK da **Groq Cloud** executando o **Llama 3.1 8B Instant**. A escolha baseou-se em:

1. **Latência Ultra-Baixa:** Inferência acelerada reduzindo o tempo de resposta a segundos.
2. **Estabilidade na PoC:** Eliminação de gargalos comerciais de infraestrutura durante a demonstração.

3. Matriz de Acertos (Sucessos Operacionais)

Durante os testes integrados via Docker, o KLA obteve os seguintes marcos:

- **Isolamento de Runtime:** O contêiner processou os dados de forma limpa, sem conflitos com o host.
- **Velocidade de Inferência Industrial:** O tempo total (leitura, envio à API e renderização no terminal) foi estável em **~4 segundos**.
- **Precisão de Reconhecimento Tático:** A IA identificou corretamente:
 1. O ataque externo por brute force (IP invasor `185.220.101.45`).
 2. A movimentação lateral/abuso de privilégio administrativo (`/bin/cat /etc/shadow`).

4. Matriz de Erros e Alucinações (Análise Crítica e Lições Aprendidas)

A maturidade de engenharia deste projeto reside na capacidade de auditar as respostas da IA, mapeando falhas conceituais graves que comprometeriam a infraestrutura se aplicadas de forma autônoma.

4.1. Caso de Falha Crítica: A Alucinação de Concessão de Privilégio

Ao sugerir mitigação para o abuso de leitura do arquivo de senhas (`/etc/shadow`), a IA sugeriu adicionar a seguinte linha ao `/etc/sudoers`:

```
admin ALL=(ALL) NOPASSWD: /bin/cat /etc/shadow
```

Análise de Impacto: Isso representa um erro catastrófico. Em vez de restringir, a IA recomendou abrir uma brecha crítica permanente: conceder permissão para leitura do arquivo de hashes de senha com privilégio máximo (`root`), sem exigir validação (`NOPASSWD`).

4.2. O Limite Lógico das Listas de Bloqueio (*Blacklists*)

A IA não compreendeu que a segurança no Linux baseia-se no **Princípio do Menor Privilégio** (*Whitelists*). Tentar contornar o problema usando negação explícita (`!/bin/cat /etc/shadow`) cria

apenas um "teatro de segurança", pois um atacante com privilégios ALL pode realizar bypasses simples:

- 1. **Leitores Alternativos:** Usar less, tail ou vim.
- 2. **Cópia de Arquivo:** Executar cp /etc/shadow /tmp/copia.txt e ler a cópia.
- 3. **Shell Interativa:** Executar sudo su - para se tornar root irrestrito e ignorar a diretiva.

4.3. Dependência de Conectividade Externa

A PoC comprovou que arquiteturas SaaS criam dependência de conectividade WAN. Em ambientes de SOC estritamente isolados (air-gapped), a arquitetura precisaria ser adaptada para rodar modelos LLM nativos locais com aceleração de GPU (ex: via Ollama/vLLM).

4.4 Controle de versão

Versão	Data	Autor	Descrição da Mudança
1.0	11/06/2026	Eduardo Gomes	Lançamento da PoC: Motor Llama 3.1, Docker, CLI
0.1	01/06/2026	Eduardo Gomes	MVP interno: Integração básica com Gemini

5. Conclusão: O Modelo Ideal de Operação SOC

O **Kensei Log Auditor (KLA)** prova que a Integração de IA reduz o **MTTD (Mean Time to Detect)** automatizando o processamento de logs brutos em tempo real.

Contudo, os testes validam o conceito de **Human-in-the-Loop**. Os modelos LLM são auxiliares de detecção e não administradores autônomos de infraestrutura. A aplicação final de regras e mitigação de vulnerabilidades sistêmicas exige obrigatoriamente a curadoria e a assinatura de um engenheiro de segurança.

Relatório emitido em conformidade técnica com as diretrizes do programa Kensei AI Foundations 2026.